



JavaScript库-jQuery

jQuery基础教程：

JS库，极大简化JS编程

1.jQuery简介：

jQuery可以将多行js代码包装为一行代码调用

jQuery简化了JS中许多复杂功能，如AJAX调用与DOM操作

主要包含以下功能：

- HTML/DOM操作
- CSS操作
- HTML事件方法
- 效果与动画
- AJAX
- 实用工具

此外，jQuery几乎为所有任务都提供了插件

2.jQuery入门：

将jQuery添加到网页：

- 在官网中下载jQuery，将该库在HTML<head>标签中引用

```
<head>
```

```
<script src="jquery-3.7.1.min.js"></script>
```

```
</head>
```

- 在CDN中引入jQuery（推荐使用Google托管，缩短加载时间）

```
<head>
```

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js">
</script>
</head>
```

3.jQuery语法：

jQuery语法是为了选择HTML元素，并对元素执行操作而设计的

基本语法是：

`$(selector).action()`

\$符号用于定义、访问jQuery

selector选择器用于查找定位HTML元素

action () 指对元素要执行的操作

例子如下：

- `$(this).hide()`，隐藏当前元素
- `$("p").hide()`，隐藏所有的<p>元素
- `$(".test").hide()`，隐藏所有带有class="test"的元素
- `$("#test").hide()`，隐藏id = "test"的元素

文件准备就绪活动：

```
$ (document) .ready (function () {
```

jQuery的方法在此处运行

```
});
```

jQuery方法都在文档就绪事件中，为了防止jQuery代码在文档加载完成之前运行等待文档完全加载并准备就绪后在执行操作是好习惯，这样也可以将JS代码放在<head>部分中

下例为失败操作：即文档加载完全之前运行方法

- 尝试隐藏尚未创建的元素
- 尝试获取尚未加载的图像的属性

提示：文档就绪事件活动可以更为缩短

```
$ (function () {
```

```
});
```

4.jQuery选择器：

jQuery选择器是该库最重要的部分

jQuery选择器：

jQuery选择器允许选择和操作html元素

jQuery选择器基于现有的CSS选择器，也有一些自定义的选择器

所有jQuery中的**选择器均以美元开头：\$()**

元素选择器：

jQuery元素选择器根据元素名称选择定位元素

\$ ("p")：选择所有的<p>元素

#id选择器：

jQuery#id选择器使用HTML标签的元素属性来选择定位元素

\$ ("#test")：选择id为text的元素

.class类选择器：

jQuery.class类选择器选择定位具有特定类的元素

\$ (".test")：选择.class类名为text的元素

组合选择器：

- \$ ("*")：选择文档中所有元素
- \$ (this)：选择当前的元素
- \$ ("p.test")：选择所有类名为test的<p>元素
- \$ ("p: first")：选择文档中的第一个p元素
- \$ ("ul li: first")：选择第一个ul元素的li元素
- \$ ("ul li: first-child")：选择每个ul元素中的第一个li元素
- \$ ("【href】")：选择所有具有href属性的元素
- \$ ("a 【target='_blank'】")：选择所有target属性等于_blank的<a>元素
- \$ ("a 【target! ='_blank'】")：选择所有target属性不等于_blank的<a>元素
- \$ (": button")：选择所有的button元素和type="button"的input元素

- `$ ("tr: even")`: 选择所有偶数行的tr元素，从0开始计数
- `$ ("tr: odd")`: 选择所有奇数行的tr元素

可以把jQuery所有的函数存放于单独文件

5.jQuery事件方法:

jQuery是专门为响应HTML页面中的事件而设计

事件:

网页可以响应的所有不同访问者的行为称之为事件

时间可以表示某件事发生的精确时刻

- 鼠标移到元素上
- 选择单选按钮
- 点击元素

常见事件:

鼠标事件	键盘事件	表单事件	文档事件
click (点击鼠标)	keypress (键盘敲击)	submit (表单提交)	load (文档加载完成时)
dblclick (双击鼠标)	keydown (键盘按下)	change (字段值修改)	resize (文档大小调整时)
mouseenter (鼠标按下)	keyup (键盘抬起)	focus (元素聚焦)	scroll (文档滚动时)
mouseleave (鼠标抬起)		blur (元素失去焦点)	unload (文档被卸载时)

事件方法的jQuery语法:

jQuery中大多数的DOM事件都有等效的jQuery方法

如: 点击时分配事件

`$ ("p").click ()`: 页面上所有段落分配点击事件

定义事件触发时应发生什么, 需要传递函数给事件

```
$ ("p").click (function () {
```

```
  行为函数
```

```
});
```

常用的jQuery事件方法：

- `$ (document) .ready ()`：在文档完全加载时执行一个函数
- `$ () .click ()`：用户点击该元素时执行函数

```
$("#p").click(function(){  
$(this).hide();  
});
```

- `$ () .dblclick ()`：用户双击该元素时执行函数

```
$("#p").dblclick(function(){  
$(this).hide();  
});
```

- `$ () .mouseenter ()`：用户鼠标进入该元素时执行函数

```
$("#p1").mouseenter(function(){  
alert("You entered p1!");  
});
```

- `$ () .mouseleave ()`：用户鼠标离开该元素时执行函数

```
$("#p1").mouseleave(function(){  
alert("Bye! You now leave p1!");  
});
```

- `$ () .mousedown ()`：用户鼠标按下时执行函数

```
$("#p1").mousedown(function(){  
alert("Mouse down over p1!");  
});
```

- `$ () .mouseup ()`：用户鼠标抬起时执行函数

```
$("#p1").mouseup(function(){  
alert("Mouse up over p1!");  
});
```

- `$ () .hover ()`：用户鼠标徘徊该元素时执行函数，等同于mouseenter和mouseleave

```

$("#p1").hover(function(){
    alert("You entered p1!");
},
function(){
    alert("Bye! You now leave p1!");
});

```

- `$().focus()`：当表单字段获得焦点时执行函数

```

$("input").focus(function(){
    $(this).css("background-color", "#cccccc");
});

```

- `$().blur()`：当表单字段失去焦点时执行函数

```

$("input").blur(function(){
    $(this).css("background-color", "#ffffff");
});

```

- `$().on()`：为所选元素附加一个或多个事件处理程序

一个事件：

```

$("p").on("click", function(){
    $(this).hide();
});

```

多个事件：

```

$("p").on({
    mouseenter: function(){
        $(this).css("background-color", "lightgray");
    },
    mouseleave: function(){
        $(this).css("background-color", "lightblue");
    },
    click: function(){
        $(this).css("background-color", "yellow");
    }
});

```

```
}  
});
```

jQuery动画效果:

1.效果-显示和隐藏:

使用jQuery，可以使用hide（）和show（）方法隐藏和显示HTML元素

语法:

```
$(selector).hide(speed, callback);  
$(selector).show(speed, callback);
```

例子:

```
$("#hide").click(function(){  
    $("p").hide();  
});  
$("#show").click(function(){  
    $("p").show();  
});
```

可选的速度参数，可以取值“快、慢、毫秒”，下例展示速度参数

```
$("#button").click(function(){  
    $("p").hide(1000);  
});
```

2.效果-切换toggle（）:

可以使用该方法toggle（）在隐藏于显示元素之间切换

语法:

```
$(selector).toggle(speed, callback);
```

例子：显示元素被隐藏，隐藏元素被显示

```
$("#button").click(function(){  
    $("p").toggle();  
});
```

3.效果-淡入淡出:

使用jQuery可也是元素淡入淡出可见

具有以下淡入淡出方法

- `fadeIn ()`：用于淡入隐藏元素
- `fadeOut ()`：用于淡出显示元素
- `fadeToggle ()`：用于切换淡入淡出方法
- `fadeTo ()`：允许淡入淡出至特定的不透明度之间

fadeIn ()：淡入

用于淡入隐藏元素

语法：

```
$(selector).fadeIn(speed,callback);
```

例子：具有不同参数

```
$("#button").click(function(){  
    $("#div1").fadeIn(); //直接淡入  
    $("#div2").fadeIn("slow"); //缓慢淡入  
    $("#div3").fadeIn(3000); //三秒内完成淡入  
});
```

fadeOut ()：淡出

用于淡出显示元素

语法：

```
$(selector).fadeOut(speed,callback);
```

例子：具有不同参数

```
$("#button").click(function(){  
    $("#div1").fadeOut();  
    $("#div2").fadeOut("slow");  
    $("#div3").fadeOut(3000);  
});
```

fadeToggle ()：切换

用于切换淡入淡出方法

语法：


```
$(selector).fadeToggle(speed,callback);
```

例子：

```
$("#button").click(function(){  
    $("#div1").fadeToggle(); //假如淡入  
    $("#div2").fadeToggle("slow"); //缓慢淡出  
    $("#div3").fadeToggle(3000); //三秒内完成淡入  
});
```

fadeTo ()：淡入淡出到指定透明度

允许淡入淡出至指定的不透明度（0-1之间）

语法：speed是必要的，不透明度是必要的，可选回调参数

```
$(selector).fadeTo(speed, opacity, callback);
```

例子：不同参数

```
$("#button").click(function(){  
    $("#div1").fadeTo("slow", 0.15);  
    $("#div2").fadeTo("slow", 0.4);  
    $("#div3").fadeTo("slow", 0.7);  
});
```

4.效果-滑动：

jQuery滑动方法可以使得元素上下滑动

具有以下方法为元素创建滑动效果

- slideDown ()：向下滑动
- slideUp ()：向上滑动
- slideToggle ()：切换滑动方向

slideDown ()：向下滑动

用于向下滑动元素

语法：

```
$(selector).slideDown(speed,callback);
```

例子：

```
$("#flip").click(function(){
```

```
$("#panel").slideDown();  
});
```

slideUp ()：向上滑动

用于向下滑动元素

语法：

```
$(selector).slideUp(speed,callback);
```

例子：

```
$("#flip").click(function(){  
    $("#panel").slideUp();  
});
```

slideToggle ()：切换上下滑动方向

用于切换上下滑动方向

语法：

```
$(selector).slideToggle(speed,callback);
```

例子：

```
$("#flip").click(function(){  
    $("#panel").slideToggle();  
});
```

5.效果-动画：

使用jQuery可以创建自定义动画

animate ()：方法用于创建自定义动画

语法：params时必须的CSS动画属性，其他可选

```
$(selector).animate({params},speed,callback);
```

例子：div>右移，到left属性的250px

```
$("#button").click(function(){  
    $("#div").animate({left: '250px'});  
});
```

默认情况下，所有 HTML 元素都具有静态位置，无法移动。

要操纵位置，请记住先将元素的 CSS 位置属性设置为相对、固定或绝对！

animate ()：操作多个属性

可以同时多个属性进行动画处理

例子：

```
$("#button").click(function(){
    $("#div").animate({
        left: '250px',
        opacity: '0.5',
        height: '150px',
        width: '150px'
    });
});
```

是否可以使用 `animate()` 方法操作所有 CSS 属性？

是的，几乎可以！但是，有一件重要的事情需要记住：与 `animate()` 方法一起使用时，所有属性名称都必须采用驼峰式大小写：您需要使用 `paddingLeft` 而不是 `padding-left`、`marginRight` 而不是 `margin-right` 等等。

此外，核心 jQuery 库中不包含颜色动画。

如果要为颜色添加动画，则需要从 jQuery.com 下载

[Color Animations 插件。](#)

animate ()：使用相对值：

也可以定义相对值，相对于该元素的当前值，这可以通过在值前放置 `+=` 或 `-=` 实现

例子：

```
$("#button").click(function(){
    $("#div").animate({
        left: '250px',
        height: '+=150px',
        width: '+=150px'
    });
});
```

animate ()：使用预定义值

可以将动画属性指定为“show、hide、toggle”等预定义值

例子：

```
$("#button").click(function(){  
    $("#div").animate({  
        height: 'toggle'  
    });  
});
```

animate ()：使用队列功能

一位置animate连续编写多个调用，jQuery会为这些调用创建一个内部队列，然后逐个运行调用

例子：图形依次执行不同动画

```
$("#button").click(function(){  
    var div = $("#div");  
    div.animate({height: '300px', opacity: '0.4'}, "slow");  
    div.animate({width: '300px', opacity: '0.8'}, "slow");  
    div.animate({height: '100px', opacity: '0.4'}, "slow");  
    div.animate({width: '100px', opacity: '0.8'}, "slow");  
});
```

例子：div>向右移动，然后增加文本的字体大小

```
$("#button").click(function(){  
    var div = $("#div");  
    div.animate({left: '100px'}, "slow");  
    div.animate({fontSize: '3em'}, "slow");  
});
```

6.效果-停止动画：

stop () 方法用于在动画或效果完成之前停止

stop ()：适用于jQuery所有效果

语法：

可选的stopAll参数指定是否应该清除动画队列，默认false，意味着仅停止动画，允许随后执行排队的动画，可继续

可选的goToEnd参数，指定是否立即完成当前动画，默认false

```
$(selector).stop(stopAll,goToEnd);
```

例子：无参数

```
$("#stop").click(function(){  
    $("#panel").stop();  
});
```

7.回调函数：

当前效果或动画，100%完成之后执行回调函数

jQuery回调函数：

js语句是逐行执行的，但是使用动画时，即使未完成下一行代码也可以允许，为防止发生错误，可以创建一个回调函数，保证效果完成之后在执行其他

典型语法\$(selector).hide(speed, callback)

例子：含有一个回调函数，他是隐藏效果完成后执行的函数

```
$("#button").click(function(){  
    $("#p").hide("slow", function(){  
        alert("The paragraph is now hidden");  
    });  
});
```

8.链接：

使用jQuery，可以将操作与方法连接在一起

连接允许我们在单个语句中运行多个jQuery方法（在同一个元素上）

jQuery方法链：

通过方法链，浏览器可以不必多次查询相同的元素

要连接一个动作，只需要将该动作附加到前一个动作之后

例子：将CSS、slideUp和slideDown方法连接到一起，首先p元素变成红色，然后向上滑动，向下滑动

```
$("#p1").css("color", "red").slideUp(2000).slideDown(2000);
```

如果需要，可以添加更多方法调用，但是需要注意长行代码，可能会出错

可以使用换行和缩进

```
$("#p1").css("color", "red")
    .slideUp(2000)
    .slideDown(2000);
```

jQuery会丢弃多余的空格，并将上面的各行作为一行长代码执行。

jQuery更改网页元素

9.jQuery操作DOM：

jQuery包含用于改变和操作HTML元素和属性的方法

jQuery带有一组与DOM相关的方法

DOM文档对象模型，定义了访问HTML和XML文档的标准

10.jQuery获取内容：

- text ()：设置或返回所选元素的文本内容
- html ()：设置或返回所选元素的内容，包括HTML标记
- val ()：设置或返回表单字段的值
- attr ()：设置或返回属性值

例子：演示如何使用text () 和html () 获取内容

```
$("#btn1").click(function(){
    alert("Text: " + $("#test").text()); //返回#test的文本内容
});
$("#btn2").click(function(){
    alert("HTML: " + $("#test").html()); //返回#test的内容，包含代码
});
```

例子：val () 方法获取输入字段的值

```
$("#btn1").click(function(){
    alert("Value: " + $("#test").val()); //返回#test字段输入的值
});
```

例子：获取链接中的href属性

```
$("#button").click(function(){
```

```
alert($("#w3s").attr("href"));
});
```

11.jQuery设置内容:

- text ()
- html ()
- val ()
- attr ()

例子：使用text、html、val方法设置内容

```
$("#btn1").click(function(){
$("#test1").text("Hello world!");
});
$("#btn2").click(function(){
$("#test2").html("<b>Hello world!</b>");
});
$("#btn3").click(function(){
$("#test3").val("Dolly Duck");
});
```

例子：使用attr () 方法设置链接中的href属性的值

设置一个属性：

```
$("#button").click(function(){
$("#w3s").attr("href", "https://www.w3schools.com/jquery/");
});
```

设置多个属性：href和title

```
$("#button").click(function(){
$("#w3s").attr({
"href" : "https://www.w3schools.com/jquery/",
"title" : "W3Schools jQuery Tutorial"
});
});
```

12.获取内容后的回调函数：

例子：text、html、val方法的回调函数，包含两个参数：所选元素列表中当前元素的索引和原始值，可以从函数中返回要用作新值的字符串。

```
$("#btn1").click(function(){
    $("#test1").text(function(i, origText){
        return "Old text: " + origText + " New text: Hello world!
        (index: " + i + ")";
    });
});

$("#btn2").click(function(){
    $("#test2").html(function(i, origText){
        return "Old html: " + origText + " New html: Hello <b>world!</b>
        (index: " + i + ")";
    });
});
```

例子：attr () 方法的回调函数，回调函数包含两个参数：所选元素列表中当前元素的索引和原始属性值，也可以从函数中返回要用做新属性值的字符串。

```
$("#button").click(function(){
    $("#w3s").attr("href", function(i, origValue){
        return origValue + "/jquery/";
    });
});
```

13.jQuery添加元素：

使用jQuery方法添加新内容：

- append ()：在所选元素的末尾插入内容
- prepend ()：在所选元素的开头插入内容
- after ()：在选定元素后插入内容
- before ()：在选定元素之前插入内容

append () 方法插入内容：

在所选的HTML元素末尾插入内容

例子：

```
$("#p").append("Some appended text.");
```

prepend () 方法插入内容：

在所选的HTML开头插入内容

例子：

```
$("#p").prepend("Some prepended text.");
```

append () 和prepend () 添加新元素：

例子：创建新元素，text/HTML、jQuery和jsDOM创建，使用append () 方法将新元素附加到文本中，方法同样使用prepend ()

```
function appendText() {  
    var txt1 = "<p>Text.</p>";           // Create element with HTML  
    var txt2 = $("#<p></p>").text("Text."); // Create with jQuery  
    var txt3 = document.createElement("p"); // Create with DOM  
    txt3.innerHTML = "Text.";   
    $("#body").append(txt1, txt2, txt3); // Append the new elements  
}
```

after () 和before () 在选定的HTML元素前后插入内容：

例子：

```
$("#img").after("Some text after");
```

```
$("#img").before("Some text before");
```

after () 和before () 添加新元素：

例子：

```
function afterText() {  
    var txt1 = "<b>I </b>";           // Create element with HTML  
    var txt2 = $("#<i></i>").text("love "); // Create with jQuery  
    var txt3 = document.createElement("b"); // Create with DOM  
    txt3.innerHTML = "jQuery!";   
    $("#img").after(txt1, txt2, txt3); // Insert new elements after <img>
```

```
}
```

14.jQuery删除元素:

主要两种jQuery方法来删除元素与内容

- `remove ()`: 删除选定的元素 (及其子元素)
- `empty ()`: 从选定的元素中删除子元素

remove ():

例子: 删除div1及其子元素

```
$("#div1").remove();
```

empty ():

例子: 删除div1的子元素

```
$("#div1").empty();
```

过滤要移除的元素:

`remove ()` 方法可以接受一个函数, 允许过滤要删除的元素

该参数可以是任何jQuery选择器语法

例子: 删除<p>带有class="test"的元素

```
$("p").remove(".test");
```

例子: 删除所有带有class="test"和class="demo"的<p>元素

```
$("p").remove(".test, .demo")
```

15.jQuery获取和设置CSS类:

jQuery操作CSS的主要方法:

- `addClass ()`: 向所选元素添加一个或多个类
- `removeClass ()`: 在选定元素中删除一个或多个类
- `toggleClass ()`: 在所选元素中添加或删除类之间切换
- `css ()`: 设置或返回样式属性

addClass ():

例子: 给不同元素添加类属性、添加类时也可以选择多个元素

```
$("button").click(function(){
```

```
  $("h1, h2, p").addClass("blue");
```

```
$("#div").addClass("important");  
});
```

例子：为选定元素添加多个类

```
$("#button").click(function(){  
$("#div1").addClass("important blue");  
});
```

removeClass ():

例子：从不同元素中删除特定的类

```
$("#button").click(function(){  
$("#h1, h2, p").removeClass("blue");  
});
```

toggleClass ():

可以在所选元素中添加、删除类之间切换

例子：

```
$("#button").click(function(){  
$("#h1, h2, p").toggleClass("blue");  
});
```

css ():

设置或返回所选元素的一个或多个样式属性

001.返回CSS属性：

语法：

```
css("propertyname");
```

例子：返回第一个匹配元素的背景颜色值

```
$("#p").css("background-color");
```

002.设置1个CSS属性：

语法：

```
css("propertyname","value");
```

例子：设置所有匹配元素的背景颜色

```
$("#p").css("background-color", "yellow");
```

003.设置多个CSS属性：

语法:

```
css({"propertyname":"value","propertyname":"value",...});
```

例子: 为所有匹配的元素设置背景颜色和字体大小:

```
$("p").css({"background-color": "yellow", "font-size": "200%"});
```

16.jQuery尺寸

Margin

Border

Padding

元素 Padding Border Margin

width】

innerWidth 】

outerWidth 】

outerWidth (true) 】

处理尺寸的重要方法:

- width () : 设置或返回元素的宽度, 不包括填充、边框、边距
- height () : 设置或返回元素的高度, 不包括填充、边框、边距
- innerWidth () : 返回元素的宽度, 包括填充
- innerHeight () : 返回元素的高度, 包括填充
- outerWidth () : 返回元素的宽度, 包括填充和边框
- outerHeight () : 返回元素的高度, 包括填充和边框

width () 和height () : 返回指定元素的高度与宽度:

```
$("button").click(function(){  
var txt = "";  
txt += "Width: " + $("#div1").width() + "</br>";  
txt += "Height: " + $("#div1").height();  
$("#div1").html(txt);  
});
```

innerWidth () 和innerHeight () 返回元素的宽度与高度, 包括填充

例子:

```

$("button").click(function(){
var txt = "";
txt += "Inner width: " + $("#div1").innerWidth() + "<br>";
txt += "Inner height: " + $("#div1").innerHeight();
$("#div1").html(txt);
});

```

outerWidth () 和outerHeight () 返回元素的宽度与高度，包括填充与边框

例子：

```

$("button").click(function(){
var txt = "";
txt += "Outer width: " + $("#div1").outerWidth() + "<br>";
txt += "Outer height: " + $("#div1").outerHeight();
$("#div1").html(txt);
});

```

outerWidth (true) 和outerHeight (true) 返回元素的宽度与高度，包括填充与边框与边距

例子：

```

$("button").click(function(){
var txt = "";
txt += "Outer width (+margin): " + $("#div1").outerWidth(true) + "<br>";
txt += "Outer height (+margin): " + $("#div1").outerHeight(true);
$("#div1").html(txt);
});

```

jQuery更多的width和height ()

例子：以下示例返回HTML文档和浏览器窗口的宽度和高度

```

$("button").click(function(){
var txt = "";
txt += "Document width/height: " + $(document).width();
txt += "x" + $(document).height() + "\n";

```

```
txt += "Window width/height: " + $(window).width();  
txt += "x" + $(window).height();  
alert(txt);  
});
```

jQuery遍历树

17.jQuery遍历：

遍历即移动定位，根据html元素与其他元素的关系，选择元素，从一个选择开始然后移动，直到找到所需的元素

```
<div>  
  <ul>  
    <li>  
      <span>  
    </span>  
    </li>  
    <li>  
      <b>  
    </li>  
  </ul>  
</div>
```

DOM树使用jQuery遍历，可以从当前元素，向上、向下、侧面移动，这个移动叫做遍历DOM树

18.jQuery遍历DOM树-祖先

遍历DOM树的三个有用方法

- `parent ()`：返回所选元素的直接父元素，仅遍历DOM树的单个级别
- `parents ()`：返回所选元素的所有祖先元素，直到文档的根元素
- `parentsUntil ()`：返回两个给定参数之间的所有祖先元素

`parent ()`：返回所选元素的直接父元素

例子：返回每个>的直接父元素

```
$(document).ready(function(){
```

```
$("#span").parent();  
});
```

parents ()：返回所选元素的所有祖先元素，可选指定元素参数

例子：返回所有span>元素的所有祖先

```
$(document).ready(function(){  
$("#span").parents();  
});
```

例子：可以使用可选参数来过滤祖先的搜索，下例来返回所有span>元素的所有祖先ul>元素

```
$(document).ready(function(){  
$("#span").parents("ul");  
});
```

parentsUntil ()：返回两个给定参数之间的所有祖先元素

例子：返回span>和div>元素之间的所有祖先元素

```
$(document).ready(function(){  
$("#span").parentsUntil("div");  
});
```

19.jQuery遍历DOM树-后代：

两个常用的方法是

- children ()：返回所选元素的所有直接子元素
- find ()：返回所选元素的后代，直到最后一个后代

children ()：返回所选元素的所有直接子元素

例子：返回每个div元素的所有直接子元素

```
$(document).ready(function(){  
$("#div").children();  
});
```

例子：可以使用可选参数来过滤子项的搜索，下例用于返回所有div子元素中，具有first类名的直接的子元素p>，

```
$(document).ready(function(){
```

```
$("#div").children("p.first");  
});
```

find ()：返回所选元素的所有后代元素，直到最后

例子：返回所有div>元素的后代元素span>

```
$(document).ready(function(){  
$("#div").find("span");  
});
```

例子：返回div>元素的所有后代

```
$(document).ready(function(){  
$("#div").find("*");  
});
```

20.jQuery遍历DOM树-兄弟：

使用jQuery方法可以在DOM书中横向遍历查找元素

- siblings ()：返回所选元素的所有同级元素
- next ()：返回所选元素的下一个同级元素
- nextAll ()：返回所选元素的所有下一个同级元素
- nextUntil ()：返回两个给定参数之间的所有下一个同级元素
- prev () ——》
- prevAll () ——》》 与上面的方法相同，但功能相反，返回前一个同级元素
- prevUntil () ——》

siblings ()：返回所选元素的所有同级元素

例子：返回h2的所有同级元素

```
$(document).ready(function(){  
$("#h2").siblings();  
});
```

例子：可以使用可选的参数来过滤兄弟结点的搜索，下例返回所有h2同级的p元素

```
$(document).ready(function(){  
$("#h2").siblings("p");
```



```
});
```

next ()：返回所选元素的下一个同级元素

例子：返回h2的下一个同级元素

```
$(document).ready(function(){  
    $("h2").next();  
});
```

nextAll ()：返回所选元素的所有下一个同级元素

```
$(document).ready(function(){  
    $("h2").nextAll();  
});
```

nextUntil ()：返回两个给定参数之间的所有下一个同级元素

例子：返回h2和h6元素之间的所有同级元素

```
$(document).ready(function(){  
    $("h2").nextUntil("h6");  
});
```

22.jQuery遍历DOM-过滤

最基本的过滤方法：first ()、last ()、eq ()，他们允许根据元素在一组元素中的位置选择特定元素

其他过滤方法：filter ()、not ()，允许选择符合或不符合特定条件的元素

- first ()：返回指定元素的第一个元素
- last ()：返回指定元素的最后一个元素
- eq ()：返回指定元素中具有特定索引号的元素
- filter ()：允许指定一个条件，不符合的元素从选择中删除，符合的元素被返回
- not ()：返回所有不符合条件的元素，与filter () 相反

first ()：返回指定元素的第一个元素

例子：返回第一个div>元素

```
$(document).ready(function(){  
    $("div").first();  
});
```

last ()：返回指定元素的最后一个子元素

例子：返回最后一个div>元素

```
$(document).ready(function(){  
    $("div").last();  
});
```

eq ()：返回选定元素中具有特定索引号的元素

索引号从0开始

例子：选择第二个p>元素（索引号1）

```
$(document).ready(function(){  
    $("p").eq(1);  
});
```

filter ()：允许指定条件，选择符合条件的元素，不符合的会从选择中删除

例子：选择所有类名为intro的p>元素

```
$(document).ready(function(){  
    $("p").filter(".intro");  
});
```

not ()：返回所有不符合条件的元素

例子：返回所有不具有类名为intro的p>元素

```
$(document).ready(function(){  
    $("p").not(".intro");  
});
```

jQuery的AJAX方法

23.AJAX简介：

AJAX是一种与服务器交换数据并且更新网页部分内容的技术，无需重新加载整个页面

AJAX = 异步JS + XML

jQuery提供了几种方法来实现AJAX功能，例如使用HTTP Get和HTTP Post从服务器请求文本、HTML、XML、JSON等，也可以将外部数据直接加载到网页内的指定元素中

24.jQuery-AJAX的load ()

该方法可以从服务器加载数据并将返回的数据放入所选元素中

语法：必选的URL、可选的数据参数和回调函数

```
$(selector).load(URL,data,callback);
```

例子：将指定文件的内容加载到特定的div>元素中

```
$("#div1").load("demo_test.txt");
```

例子：可以向URL参数中添加Jquery选择器，下例将文件中的id="p1"元素的内容加载到特定的div>元素中

```
$("#div1").load("demo_test.txt #p1");
```

可选的回调函数，指定方法完成时候运行的函数，回调函数可以有不同的参数

- responseTxt，如果调用成功则包含结果内容
- statusTxt，包含呼叫状态
- xhr，包含XMLHttpRequest对象

下例值load方法加载完成后，执行回调函数来显示警示框

```
$("button").click(function(){  
    $("#div1").load("demo_test.txt", function(responseTxt, statusTxt, xhr){  
        if(statusTxt == "success")  
            alert("External content loaded successfully!");  
        if(statusTxt == "error")  
            alert("Error: " + xhr.status + ": " + xhr.statusText);  
    });  
});
```

25.jQuery-AJAX的get () 与post ():

jQuery 的get () 和post () 方法用于通过HTTP GET或POST请求从服务器请求数据

HTTP请求：GET和POST

- GET：从指定资源请求数据，基本用于从服务器获取数据，可能会返回缓存数据

- POST：向指定资源提交要处理的数据，可也用于从服务器获取数据，但是POST不会缓存数据，通常直接与请求一起发送数据

\$.get ()：用于使用HTTP GET请求从服务器获取数据

语法：必选的URL参数，可选的回调函数

\$.get (URL,callback);

例子：使用get方法从服务器上的文件中检索数据

URL时请求的资源地址、第一个回调参数来保存请求页面内容、第二个回调参数保存请求状态

```
$("#button").click(function(){  
$.get("demo_test.asp", function(data, status){  
alert("Data: " + data + "\nStatus: " + status);  
});  
});
```

\$.post ()：用于使用HTTP POST请求从服务器请求数据

语法：必选的URL参数，可选的数据参数指定与请求一起发送的数据、可选的回调函数

\$.post (URL, data, callback)

例子：使用post方法随请求一起发送一些数据

```
$("#button").click(function(){  
$.post("demo_test_post.asp",  
{  
name: "Donald Duck",  
city: "Duckburg"  
},  
function(data, status){  
alert("Data: " + data + "\nStatus: " + status);  
});  
});
```

请求的ASP文件如下

```

<%
dim fname,city
fname=Request.Form("name")
city=Request.Form("city")
Response.Write("Dear " & fname & ". ")
Response.Write("Hope you live well in " & city & ".")
%>

```

26.jQuery与其他框架的使用：

jQuery使用\$，其他框架使用相同的快捷符号，可能会报错

noConflict () 方法，释放对\$快捷富豪的保留，其他脚本去使用它

仍然可以使用jQuery，只需要写全名jQuery来代替\$

```

$.noConflict();

jQuery(document).ready(function(){
    jQuery("button").click(function(){
        jQuery("p").text("jQuery is still working!");
    });
});

```

创建自定义快捷符：使用该方法返回对jQuery的引用，可以将其保存在变量中供以后使用

```

var jq = $.noConflict();

jq(document).ready(function(){
    jq("button").click(function(){
        jq("p").text("jQuery is still working!");
    });
});

```

27.jQuery过滤器：

使用jQuery可以过滤或搜索特定元素

过滤表格：

对表格中的项目进行不区分大小写的搜索

```

<script>

```

```
$(document).ready(function(){
    $("#myInput").on("keyup", function() {
        var value = $(this).val().toLowerCase();
        $("#myTable tr").filter(function() {
            $(this).toggle($(this).text().toLowerCase().indexOf(value) > -1)
        });
    });
});
</script>
```

示例说明：我们使用 jQuery 循环遍历每个表行，以检查是否有任何文本值与输入字段的值匹配。该方法隐藏与搜索不匹配的 `toggle()` 行 ()。我们使用 DOM 方法将文本转换为小写，这使得搜索不区分大小写（允许搜索“john”、“John”甚至“JOHN”）。

`display:none`toLowerCase()

过滤列表：

对列表中的项目执行不区分大小写的搜索

过滤任何内容